Beginner-Friendly Month 1 — Computer Architecture & Assembly (24-Day Plan, v2)

This revision tightens **clarity, friendliness, and sufficiency of resources**. Each day now includes: **Purpose • Prereqs • Quick Glossary • Learn (curated links) • Hands-On (stepwise) • Done-When checklist • Troubleshooting • Optional Plan-breaker**. Links are authoritative or widely vetted.

Target: **x86-64 (Intel syntax) on Linux/WSL/macOS** with **NASM + GCC/Clang + Binutils + GDB**. AArch64 tips included where relevant.

Setup (once, 30–60 min)

- Linux/WSL: `sudo apt update && sudo apt install build-essential gcc g++ make gdb binutils nasm strace linux-tools-common`
- macOS: `brew install llvm nasm binutils` (use `lldb` and `gobjdump/objdump`).
- Verify: `nasm -v`, `objdump -v`, `readelf --version`, `gdb --version`, `perf --version` (Linux).
- Create folders: `~/month1-arch/{week1-repr,week2-asm,week3-cache,week4-os}` and a `journal.md`.

Week 1 — Representation & Execution Model (Days 1–6)

Day 1 — Bits & Two’s Complement

Purpose: Understand integer storage and overflow.
Prereqs: C basics (types, printf).
Glossary: bit, byte, two’s complement, sign bit, overflow.

Learn - Two’s complement overview (gentle): https://en.wikipedia.org/wiki/Two%27s_complement
- Bit twiddling reference (browse): https://graphics.stanford.edu/~seander/bithacks.html

Hands-On (stepwise) 1) Write `char* int32_to_bin(int32_t x, char out[35])` → prints 32 bits grouped `8*4`. Provide 2 examples in `main`.
2) Implement `int32_t saturating_add(int32_t a,int32_t b)`; unit tests for `INT_MAX+1`, `INT_MIN-1`, randoms.
3) Explain in `journal.md`: why two’s complement simplifies hardware.

Done-When: prints correct bit patterns; unit tests all pass; short write-up added.
Troubleshooting: watch sign-extension when shifting; use unsigned for masking.

Plan-breaker (optional, 10m): Hand-convert three integers to two's complement and verify with your function.

Day 2 — Endianness & Layout

Purpose: See how bytes are ordered and how struct padding works.
Glossary: little/big endian, alignment, padding, `offsetof`.

Learn - Endianness primer: https://en.wikipedia.org/wiki/Endianness
- `xxd(1)`: https://man7.org/linux/man-pages/man1/xxd.1.html

Hands-On 1) Define `struct Record{ char a; int b; short c; };` Print `sizeof`, `offsetof` each field.
2) Allocate one `Record`, fill fields, `xxd -g1 -C` the memory; annotate which byte is which.
3) Implement `void to_be32(uint32_t x, uint8_t out[4])` and `uint32_t from_be32(const uint8_t in[4])`; test with `0x12345678`.

Done-When: offsets and dump match your diagram; conversions round-trip.
Troubleshooting: compile with `-Wall -Wextra`; mind UB if reading padding.

Plan-breaker: Parse a 20-byte IPv4 header (RFC 791 §3.1) fields from a hex string.

Day 3 — Floating-Point (just enough)

Purpose: Build intuition for IEEE-754 and common pitfalls.
Glossary: sign/exponent/mantissa, rounding, ULP.

Learn - Floating-Point Guide: https://floating-point-gui.de/
- Visualize bits: https://float.exposed/

Hands-On 1) `uint32_t fbits(float f)` via `memcpy`; print binary layout of `0.1f`.
2) Compare `0.1f+0.2f` vs `0.3f`; print both bit patterns; short explanation of inequality.

Done-When: you can explain why 0.1 is not exact in binary.
Troubleshooting: don't use `*(uint32_t*)&f`; use `memcpy` to avoid aliasing UB.

Plan-breaker: Show catastrophic cancellation with `sqrt(x+1)-sqrt(x)` and a stable algebraic form.

Day 4 — Logic & ALU Basics

Purpose: Relate bitwise ops to arithmetic; see a CPU's cycle conceptually.
Glossary: combinational vs sequential logic, carry, register, PC.

Learn - Instruction cycle: https://en.wikipedia.org/wiki/Instruction_cycle
- Nand2Tetris (Part I intros): https://www.nand2tetris.org/

Hands-On 1) `int bitwise_add(int a,int b)` using only `^`, `&`, `<<`; compare with `a+b`.
2) Paper CPU: write 5 fake instructions (LOAD, ADD, JNZ, HALT) and trace PC/flags for a short program.

Done-When: bitwise add matches `+`; trace table is consistent.
Troubleshooting: loop until carry==0; careful with signed vs unsigned shifts.

Plan-breaker: Implement 4-bit ripple adder in code and show intermediate carries.

Day 5 — Flags & Condition Codes

Purpose: Observe ZF/CF/OF in action.
Glossary: ZF, CF, OF, sign-extension.

Learn - Intel SDM Vol.1 (flags intro): https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- RFLAGS quick ref: https://wiki.osdev.org/FLAGS_register

Hands-On 1) Write a small C function that does `a+b`, `a-b`, `a^b`; compile with `-O0`, step in `gdb`, inspect flags after each instruction.
2) Trigger overflows: `0x7FFFFFFF+1`, `0x80000000-1`.

Done-When: you can predict which flags each ALU op changes.
Troubleshooting: use `si` in `gdb`; display `$eflags` on each step.

Plan-breaker: Hand-annotate flags for a straight-line asm snippet before running.

Day 6 — Project: Bit-Toolkit (scaffolded)

Purpose: Combine representation skills into a practical tool.
Learn: `getopt` pattern (optional): `man 3 getopt`.

Starter Layout - `bitkit.c` with subcommands: - `bin2hex <bin>` → prints `0x..` - `hex2bin <hex>` → prints grouped binary - `twos <int>` → prints 32-bit two's complement - `Makefile` with `-Wall -Wextra -O2 -g`

Hands-On 1) Implement parsing; reject invalid characters clearly.
2) Write 6 tiny tests; capture expected outputs in comments.

Done-When: commands work per examples; invalid input paths tested.

Plan-breaker: Support underscores in binary (`1010_0001`).

Week 2 — Assembly Fundamentals (Days 7–12)

Day 7 — Registers & Disassembly

Purpose: Map C to assembly; see registers and prologue/epilogue.
Glossary: caller/callee-saved, prologue, epilogue.

Learn - x86-64 overview: https://en.wikipedia.org/wiki/X86-64
- (ARM64 alt) AAPCS64: https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst

Hands-On 1) `int add(int a,int b){return a+b;}` → `gcc -O0 -S`; annotate arg registers and return.
2) `gcc -O2 -S` and compare; note differences.

Done-When: you can identify where args/return live at both `-O0` and `-O2`.

Plan-breaker: Do the same on ARM64 and list 3 differences (x0..x7, FP/LR, prologue style).

Day 8 — Assembler & Object Files

Purpose: Build a minimal asm program and inspect the object.
Glossary: section, symbol, relocation.

Learn - `objdump(1)`: https://man7.org/linux/man-pages/man1/objdump.1.html
- `readelf(1)`: https://man7.org/linux/man-pages/man1/readelf.1.html
- `nm(1)`: https://man7.org/linux/man-pages/man1/nm.1.html

Hands-On 1) `prog.asm` returns exit code 42; `nasm -f elf64 prog.asm && ld -o prog prog.o`.
2) Inspect with `objdump -d -Mintel prog`; label `.text` and the entry point; list symbols via `nm`.

Done-When: you can match instructions to bytes and point to each section.

Plan-breaker: Predict the `objdump` output (high-level) before running; compare.

Day 9 — System V AMD64 ABI

Purpose: Build ABI-correct functions.
Glossary: caller/callee-saved regs, red zone, stack alignment.

Learn - psABI: https://gitlab.com/x86-psABIs/x86-64-ABI

Hands-On 1) `add.asm` implements `int add(int,int)`; call from C; step with `gdb` to verify prologue/epilogue.
2) Add `mul2(int)` and a function that clobbers caller-saved regs; confirm saved/restored state.

Done-When: `check_abi.c` tests pass; `gdb` shows correct stack alignment.

Plan-breaker: Implement `add` without touching the stack; justify ABI safety in comments.

Day 10 — Branches & `cmov`

Purpose: Tradeoffs between branchy and branchless code.
Glossary: branch prediction, mispredict, `cmov`.

Learn - Agner Fog manuals: https://www.agner.org/optimize/
- Cloudflare primer: https://blog.cloudflare.com/branch-prediction/

Hands-On 1) `int min_branchy(int a,int b)` vs `int min_branchless(int a,int b)` using `cmov`; benchmark at N=1e7.
2) Flip input distribution (1% vs 50% taken); record break-even point.

Done-When: you can state when branchless wins/loses on your CPU.

Plan-breaker: Replace a loop branch with `cmov`/arith and verify speed.

Day 11 — Addressing & LEA

Purpose: Master effective addresses and `lea`.
Glossary: base+index*scale+disp, `lea`.

Learn - Intel SDM (addr modes): https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- `lea` tricks: https://en.wikibooks.org/wiki/X86_Assembly/LEA

Hands-On 1) Scalar `float dot_product(const float* a,const float* b,int n)` in asm; compare to C.
2) AoS vs SoA arrays: implement both and time; explain the stride effect.

Done-When: numeric equality holds; timing difference is explained.

Plan-breaker: Use `lea` to replace simple `add/shl` arithmetic and show equal or better perf.

Day 12 — Project: ASM Library + C Harness

Purpose: Package multiple asm routines with tests.
Learn: NASM manual (index): https://www.nasm.us/xdoc/2.16.01/html/nasmdoc0.html

Hands-On - Implement `add`, `my_strlen`, `dot_product` in asm; provide `main.c` tests; `Makefile` with `-O2 -g -fno-omit-frame-pointer`.
- Produce `objdump -d -Mintel` annotated listing and a `gdb` stepping transcript.

Done-When: tests pass; annotated disassembly committed.

Plan-breaker: Change C argument order via a thin shim; keep asm unchanged.

Week 3 — Memory & Performance (Days 13–18)

Day 13 — Cache Basics

Purpose: Feel spatial/temporal locality and conflict misses.
Glossary: cache line, associativity, locality, conflict/capacity miss.

Learn - Drepper: https://www.akkadia.org/drepper/cpumemory.pdf
- CS:APP site (cache lab context): https://csapp.cs.cmu.edu/

Hands-On 1) 2D array traversal: row vs column; time both; annotate line size assumption.
2) Size sweep beyond L1/L2; record timing changes.

Done-When: row-major is faster (explain why); size sweep shows step changes.

Plan-breaker: Use power-of-two sizes to surface conflict misses; hypothesize mapping.

Day 14 — Branch Prediction

Purpose: Correlate mispredicts with runtime.
Glossary: predictor, history, `perf stat`.

Learn - Cloudflare primer: https://blog.cloudflare.com/branch-prediction/
- perf examples: https://www.brendangregg.com/perf.html

Hands-On 1) Branchy vs branchless `min()` under skewed distributions; `perf stat` cycles/branches/branch-misses.
2) Sort inputs to change predictability; measure improvement.

Done-When: you can explain deltas via branch-miss rate.

Plan-breaker: Try `cmov` where predictable branches dominate; note regressions.

Day 15 — Alignment

Purpose: Quantify alignment penalties.
Glossary: alignment, `posix_memalign`, cache line boundary.

Learn - `posix_memalign(3)`: https://man7.org/linux/man-pages/man3/posix_memalign.3.html
- C alignment keywords: https://en.cppreference.com/w/c/language/_Alignas

Hands-On 1) Allocate aligned buffers (64-byte); throughput test vs +1-byte misalignment.
2) Print addresses; verify alignment numerically.

Done-When: aligned is measurably faster; you can explain why.

Plan-breaker: Align to page (4096) and compare again.

Day 16 — Atomics & x86-TSO (conceptual)

Purpose: See data races and the fix.
Glossary: TSO, happens-before, atomic.

Learn - x86-TSO overview (Sewell): https://www.cl.cam.ac.uk/~pes20/weakmemory/cacm.pdf
- Memory barriers (Linux): https://www.kernel.org/doc/Documentation/memory-barriers.txt

Hands-On 1) Broken counter with 4 threads (no atomics) → wrong totals; then fix with `stdatomic.h`.
2) Journal: describe the happens-before relation you relied on.

Done-When: fixed version is stable across runs.

Plan-breaker: Insert a sleep/yield to amplify race; show fix still holds.

Day 17 — Perf Tools

Purpose: Read useful counters and act on them.
Glossary: `perf stat`, events, sampling vs counting.

Learn - `perf stat`: https://man7.org/linux/man-pages/man1/perf-stat.1.html
- `perf record/report`: https://man7.org/linux/man-pages/man1/perf-record.1.html

Hands-On 1) Measure your Week-2 functions: cycles/op, branches, branch-misses, LLC-loads/misses.
2) Choose **one** code change from these metrics and re-measure.

Done-When: you can justify the change using the counters.

Plan-breaker: Explain one misleading metric and how you'd cross-check.

Day 18 — Project: Cache-Aware Dot Product (scaffolded)

Purpose: Apply blocking to improve locality with minimal code.
Learn: Loop blocking intro: https://en.wikipedia.org/wiki/Loop_nest_optimization#Blocking

Hands-On - Start from scalar `dot_product`; add a block size `B` (e.g., 64) and process in chunks `for (i=0;i<n;i+=B)`; keep changes <10 LoC.
- Plot (or tabulate) size vs throughput before/after; write 5 bullets explaining gains.

Done-When: measurable improvement; explanation ties to cache lines.

Plan-breaker: Try two `B` values (fits in L1 vs spills to L2) and compare.

Week 4 — OS Interface, Binaries, Integration (Days 19–24)

Day 19 — Raw Syscalls (no libc)

Purpose: Cross the user-kernel boundary directly.
Glossary: syscall number, ABI, file descriptor.

Learn - `write(2)`: https://man7.org/linux/man-pages/man2/write.2.html
- `syscall(2)`: https://man7.org/linux/man-pages/man2/syscall.2.html
- `strace(1)`: https://man7.org/linux/man-pages/man1/strace.1.html

Hands-On 1) `sys_write` and `sys_exit` in asm; small C harness prints a line using them.
2) If it fails, debug **only** with `strace -f -tt -y` and fix.

Done-When: output appears; strace shows expected syscalls.

Plan-breaker: Use `writev` to print two buffers in one syscall.

Day 20 — ELF, PLT/GOT, Static vs Dynamic

Purpose: Understand binary formats and dynamic linking.
Glossary: section vs segment, PLT, GOT, DT_NEEDED.

Learn - Dynamic linking explainer: https://lwn.net/Articles/192624/
- ELF gABI: https://refspecs.linuxfoundation.org/elf/gabi4+/contents.html

Hands-On 1) Build dynamic and static versions; compare sizes and `readelf -d`; run `ldd`.
2) Find a PLT entry in `objdump -d`; draw call path for first call.

Done-When: you can explain three concrete tradeoffs between static and dynamic.

Plan-breaker: Measure cold start time difference (very rough) with `/usr/bin/time -p`.

Day 21 — Compiler Output (`-O0` / `-O2` / `-Os`)

Purpose: See how flags change codegen.
Glossary: inlining, unrolling, strength reduction.

Learn - Compiler Explorer: https://godbolt.org/
- `objdump(1)`: https://man7.org/linux/man-pages/man1/objdump.1.html

Hands-On 1) Compile a medium C function at `-O0/-O2/-Os`; `objdump -d -Mintel -S`; annotate diff.
2) Pick a level for capstone; justify in 5–7 sentences.

Done-When: you can point to specific optimizations visible in asm.

Plan-breaker: Compare GCC vs Clang on the same snippet; note one surprising diff.

Day 22 — Interop & API Hardening

Purpose: Ship durable C↔asm boundaries.
Glossary: clobber list, reentrancy, precondition.

Learn - psABI (reference): https://gitlab.com/x86-psABIs/x86-64-ABI
- GDB manual (as needed): https://sourceware.org/gdb/current/onlinedocs/gdb/

Hands-On 1) Wrap asm funcs in a C API; validate inputs; return error codes instead of UB.
2) Add one negative test per branch; step failures in `gdb`.

Done-When: invalid inputs are rejected early and predictably.

Plan-breaker: Fuzz a pointer/length pair and confirm you never crash.

Day 23 — Capstone: `grep-lite` (scaffolded)

Purpose: Build a minimal useful tool with syscalls + asm hot loop.

Learn: `open(2)`: https://man7.org/linux/man-pages/man2/open.2.html , `read(2)`: https://man7.org/linux/man-pages/man2/read.2.html , `close(2)`: https://man7.org/linux/man-pages/man2/close.2.html

Hands-On - CLI: `grep-lite <needle> <file>`
- Syscalls only for I/O; read chunks (e.g., 8 KiB) and scan buffer; naive substring search hot loop in asm.
- Print lines containing the needle.

Done-When: matches `grep` for simple cases; no libc I/O used.

Plan-breaker: Support `-n` (line numbers) or `-i` (case-insensitive).

Day 24 — Defense & Retrospective

Purpose: Synthesize understanding and plan next steps.

Learn: CS:APP site index: https://csapp.cs.cmu.edu/
Intel SDM portal: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Hands-On - Whiteboard: source → object → ELF → process → CPU → caches; then explain your `grep-lite` performance profile.
- List five next optimizations and what to measure first.

Done-When: you can narrate the full pipeline confidently and defend design choices.

AArch64 Variant

- Use AAPCS64: https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst
- Syscall numbers differ; mirror Day 19 using your OS's table.
- Prefer `lldb` / `llvm-objdump` on macOS.

Guardrails & Study Habits

- If stuck >20m, write the blocker → skim one resource → retry (avoid rabbit holes).
- Measure or it didn't happen; record a small table for perf tasks.
- Keep each day's "Done-When" checklist visible; tick items to maintain momentum.